

Gestion des Salles



1 Table des matières

1	Table des matières	2
2	Analyse préliminaire	3
2.1	Introduction	3
2.2	Objectifs.....	3
2.3	Planification initiale	3
2.3.1	Agile.....	3
2.3.2	Sprints courts.....	4
3	Analyse / Conception.....	5
3.1	Concept	5
3.2	MCD	6
3.3	Zoning :.....	8
3.4	Stratégie de test.....	11
3.4.1	Moyens à mettre en œuvre.....	11
3.4.2	Couverture des tests (tests exhaustifs ou non).....	11
3.4.3	Données de test à prévoir.....	12
3.4.4	Testeurs extérieurs éventuels.....	12
3.5	Planification	12
3.6	Dossier de conception	13
3.6.1	Règles Stockage.....	24
4	Réalisation.....	25
4.1	Dossier de réalisation	25
4.1.1	Authentification et gestion du profil	28
5	Annexes.....	29
5.1	Résumé du rapport du TPI / version succincte de la documentation	29
5.2	Sources – Bibliographie.....	29
5.3	Journal de travail	29
		30
5.4	Cahier des charges.....	31
5.5	Manuel d'Utilisation.....	31
5.6	Archives du projet.....	31



2 Analyse préliminaire

2.1 Introduction

Dans un contexte où l'organisation d'événements, de formations ou de réunions nécessite une gestion efficace des espaces, il devient essentiel de disposer d'un outil simple, accessible et performant pour gérer les réservations de salles. "Gestion des salles" est une application web que j'ai conçue pour faciliter la location de salles en mettant à disposition un système centralisé permettant de visualiser les disponibilités, réserver des espaces adaptés et sélectionner des services complémentaires.

Ce projet s'inscrit dans le cadre du Travail Personnel d'Initiative (TPI) et il vise à développer une solution intuitive qui simplifie la gestion logistique tout en offrant aux utilisateurs (administrateurs, clients, visiteurs) une expérience fluide et personnalisée.

L'application garantit une gestion sécurisée des données, une interface responsive accessible depuis tout type d'appareil.

2.2 Objectifs

L'objectif principal de ce projet est de concevoir et développer une application web intuitive permettant de gérer la location de salles de manière simple, efficace et sécurisée.

L'application offre une visualisation claire des salles disponibles, avec leurs caractéristiques (taille, capacité, matériel, prix), et permet une réservation rapide, y compris l'ajout de services optionnels. Un système d'authentification sécurisé permettra aux utilisateurs de se connecter selon leur rôle (administrateur, client, visiteur), chacun disposant de fonctionnalités spécifiques.

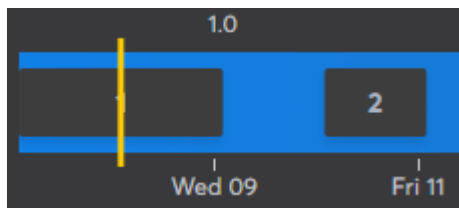
Pour les clients, la gestion des réservations sera fluide : consultation des disponibilités, ajout, modification ou suppression selon les règles définies. Pour les administrateurs, un tableau de bord affichera des statistiques sur l'utilisation des salles et des équipements. Les visiteurs auront la possibilité de consulter librement les salles disponibles, avec leurs prix et spécificités, mais devront s'inscrire ou se connecter pour pouvoir effectuer une réservation.

L'expérience utilisateur est au centre du projet, avec une interface responsive, accessible sur tout appareil. La sécurité des données est assurée via des accès contrôlés et un chiffrement des informations sensibles.

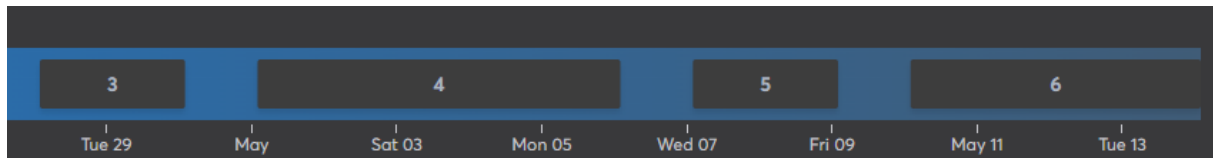
Grâce à une organisation par sprints, le projet évoluera de manière structurée et itérative, avec des tests réguliers et une documentation à jour.

2.3 Planification initiale

2.3.1 Agile



Agile 2



Agile 1

Pour garantir une gestion efficace du projet, une approche Agile. Cette méthodologie permet d'organiser le travail en sprints successifs, assurant ainsi une progression itérative et une adaptation continue aux besoins du projet. Afin de structurer cette approche, IceScrum a été choisi comme plateforme de gestion du développement. Le choix d'IceScrum repose sur plusieurs avantages fondamentaux. Tout d'abord, cet outil est parfaitement adapté aux méthodes Agile, car il permet une structuration claire du développement à travers des sprints bien définis et un backlog produit organisé. Grâce à son interface, il offre une visualisation claire des tâches, facilitant ainsi le suivi des user stories et des évolutions du projet en fonction des priorités. Cette clarté permet non seulement de maintenir une vision globale du projet, mais aussi d'ajuster la planification en fonction des retours obtenus lors des tests et des imprévus techniques.

Un autre élément clé ayant motivé le choix d'IceScrum est sa flexibilité et son évolutivité. Dans un projet informatique, les exigences peuvent évoluer rapidement en fonction des tests utilisateurs et des ajustements nécessaires à l'optimisation des fonctionnalités. IceScrum permet d'intégrer facilement ces changements, garantissant ainsi une adaptation agile du projet sans remettre en question toute la structure de développement.

Enfin, l'utilisation d'IceScrum améliore la collaboration et la communication au sein du projet. L'outil permet une documentation efficace des décisions, une meilleure organisation des tâches et une distribution claire des responsabilités. Cela assure une coordination fluide entre les différentes phases du développement et permet d'optimiser la gestion du temps et des ressources disponibles.

2.3.2 Sprints courts

La planification du projet "Gestion des salles" repose sur une organisation en sprints courts, chacun d'une durée de quelques jours à une semaine. Ce découpage a été choisi pour maintenir une dynamique de travail constante et permettre une livraison progressive des fonctionnalités.

Chaque sprint est centré sur un objectif précis (mise en place technique, gestion des profils, implémentation du modèle Salle, fonctionnalités selon le type d'utilisateur...), ce qui facilite la concentration sur des tâches ciblées.

Le choix de cette planification par sprints me permet de progresser de manière structurée, tout en conservant la flexibilité nécessaire pour m'adapter et optimiser la solution tout au long du projet.

Sprint	Dates	Sprint goal
Sprint 1	7–9 avril	Disposer d'un environnement de développement fonctionnel, d'une structure de projet claire, et d'une vision initiale complète du produit à travers des maquettes, une modélisation de données et des outils de gestion
Sprint 2	9–11 avril	Permettre aux utilisateurs de se connecter de manière sécurisée à l'application et accéder aux pages correspondant à leur rôle grâce à un système d'authentification et des guards de navigation
Sprint 3	28–30 avril	Créer et gérer les salles de manière complète dans Firestore, avec tous les attributs nécessaires, y compris les options supplémentaires et la disposition, à travers des formulaires interactifs et validés
Sprint 4	1er–6 mai	Offrir aux administrateurs un espace de gestion et de supervision complet, incluant l'édition des salles, la visualisation des réservations et l'accès à des statistiques utiles pour le suivi des activités
Sprint 5	7–9 mai	Permettre aux clients d'interagir avec l'application de manière autonome, en réservant des salles avec des options, en modifiant ou annulant leurs réservations dans le respect des règles, et en consultant leur historique
Sprint 6	12–14 mai	Offrir une interface publique claire et responsive pour les visiteurs, tout en assurant que l'application respecte les exigences du TPI et que tous les livrables sont prêts pour l'évaluation finale

3 Analyse / Conception

3.1 Concept

L'analyse des besoins et l'étude des pratiques actuelles ont permis de concevoir une plateforme répondant aux exigences de simplicité, d'ergonomie et de sécurité.

Dès l'ouverture du site, les visiteurs peuvent consulter librement la liste des salles disponibles, leurs caractéristiques (superficie, capacité, matériel inclus, prix) et visualiser leur disposition. Pour effectuer une réservation, il faut être logué, pour garantir un accès sécurisé et personnalisé.

Le système d'authentification repose sur Firebase Auth et permet une gestion précise des rôles utilisateurs. Selon leur profil, les utilisateurs accèdent à des interfaces spécifiques :

- Les clients peuvent rechercher des salles disponibles selon des dates, réserver en choisissant les options souhaitées (matériel supplémentaire, restauration), modifier ou annuler leurs réservations dans un délai défini, et consulter leur historique.
- Les administrateurs disposent d'une interface complète de gestion des salles, du contenu et des statistiques. Ils peuvent ajouter, modifier ou supprimer des salles et visualiser l'occupation mensuelle ou les options les plus demandées.
- Les visiteurs ont un aperçu public des salles, sans possibilité d'interaction, ce qui permet une découverte rapide avant inscription.



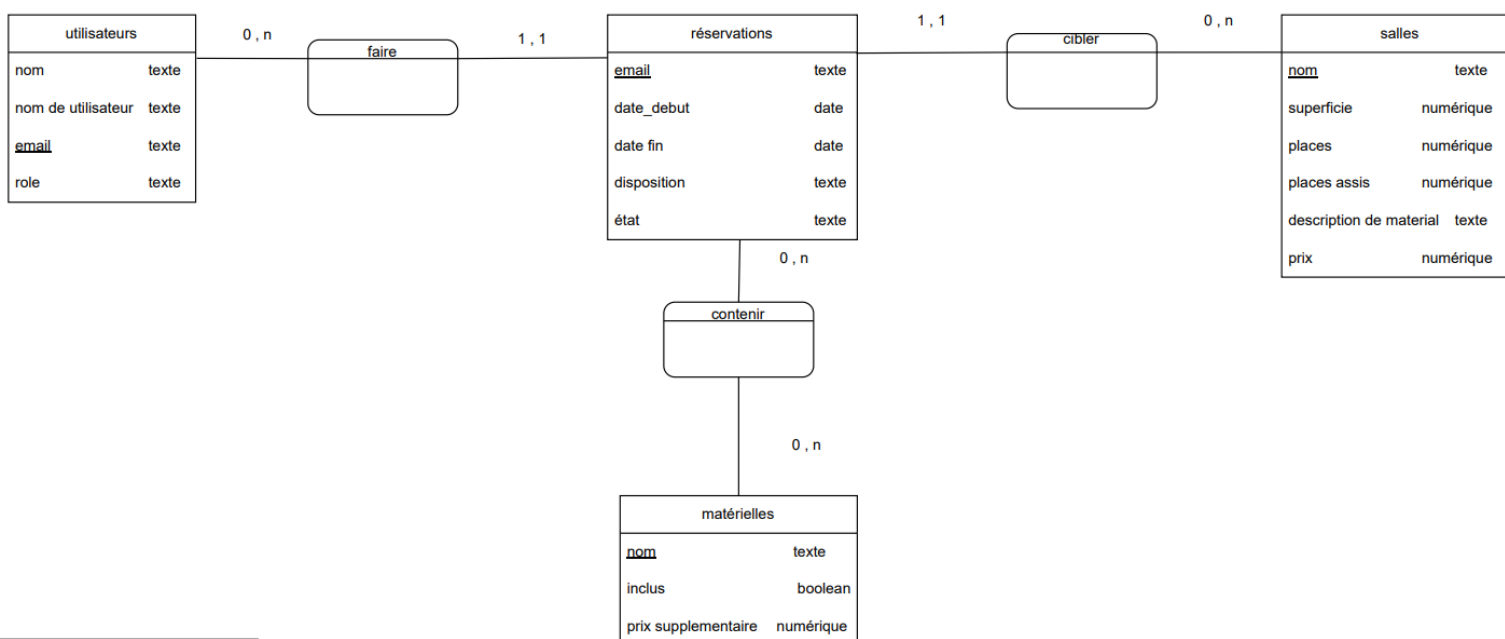
Le cœur de l'application repose sur une base de données Firebase Firestore, une solution NoSQL orientée documents. Ce type de base de données a été choisi pour sa flexibilité dans la gestion de données structurées de manière dynamique, ce qui s'avère particulièrement adapté à une application où les objets manipulés (salles, réservations, utilisateurs) peuvent varier en structure et en volume.

Grâce à Firestore, la gestion des réservations se fait de manière dynamique et en temps réel, permettant une synchronisation instantanée des données entre les utilisateurs. Cette approche facilite également le développement d'une interface responsive, fluide et accessible aussi bien sur mobile que sur desktop. Des formulaires avec validations, des tableaux de bord clairs et un design épuré viennent renforcer l'expérience utilisateur en assurant une navigation intuitive et agréable.

3.2 MCD

La structure du Modèle Conceptuel de Données (MCD) de l'application "Gestion des salles" a été conçue pour répondre aux besoins fonctionnels spécifiques de la gestion de réservation d'espaces. Ce modèle repose sur l'utilisation de Firebase Firestore, une base de données NoSQL orientée documents, offrant une flexibilité idéale pour une application dynamique avec des mises à jour en temps réel.

Le MCD comprend plusieurs entités principales, chacune jouant un rôle clé dans la structure et la logique de l'application.



Projet	Gestion des salles
Titre	Modèle conceptuel de données
Auteur	Santiago Messer
Version	1.0 (08.04.2025)

MCD 1

Utilisateurs : cette entité gère les profils des différents types d'utilisateurs (administrateur, client, visiteur). Chaque utilisateur a un nom, une adresse email et un rôle défini (administrateur, visitant ou client). Ce rôle conditionne l'accès à certaines fonctionnalités dans l'application.

Salles : il s'agit de l'entité centrale du projet. Chaque salle est décrite par un nom, une superficie, une capacité en nombre de places (avec et sans tables), une description du matériel présent et un prix de base. Ces informations sont essentielles pour permettre aux utilisateurs de sélectionner la salle la plus adaptée à leurs besoins.

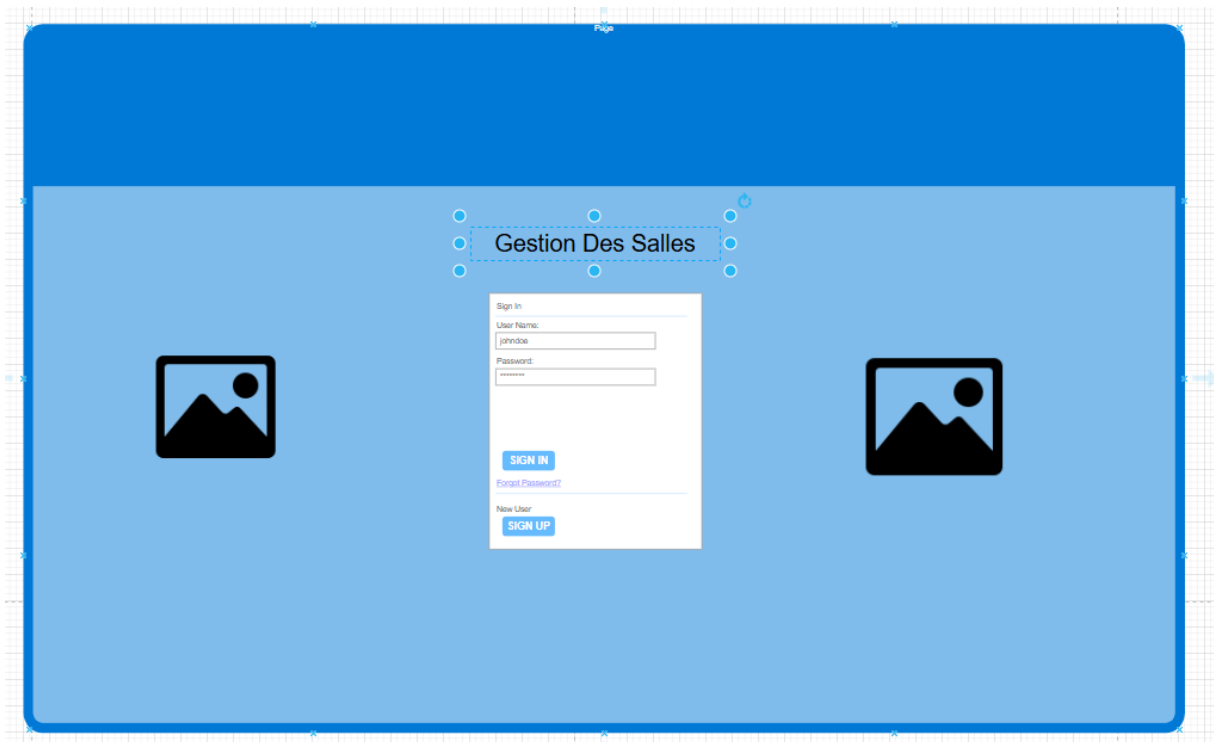
Matérielles: cette entité représente le matériel disponible en option (ex. beamer, éclairage, etc.). Chaque élément est identifié par un nom, un statut (inclus ou non) et un prix supplémentaire. Cette séparation permet une gestion fine des ressources associées à chaque salle.

Réservations : cette entité gère les demandes de location de salles. Chaque réservation est liée à un utilisateur (via son email), à une ou plusieurs salles, et contient des informations telles que la date de début, la date de fin et l'état de la réservation (confirmée, annulée, payé.) ainsi que la distribution de la salle. Ce système permet un suivi clair et fiable des engagements de location.

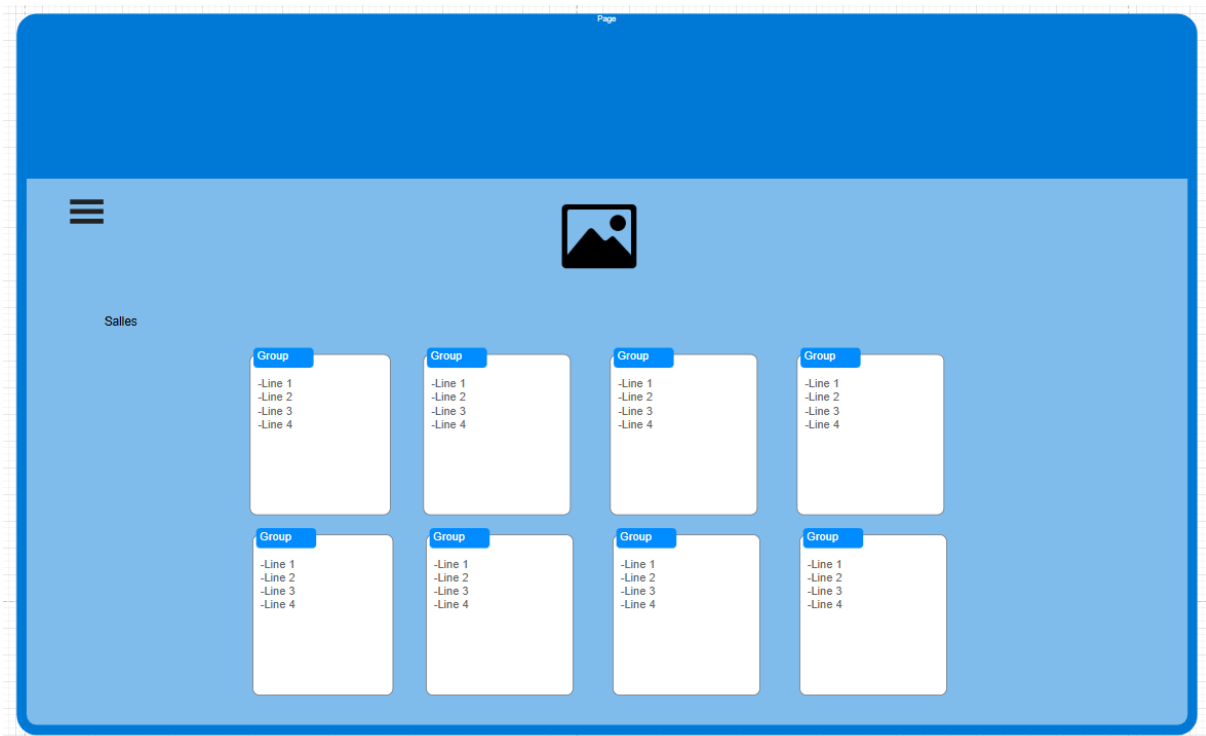
Les relations faire, contenir et cibler assurent la cohérence entre les entités, notamment en reliant les réservations aux salles et au matériel réservé. Le schéma NoSQL permet ici une certaine souplesse, en évitant les jointures complexes tout en assurant une structure logique efficace et évolutive.

Ce MCD garantit une base solide pour le développement de l'application, en assurant à la fois performance, évolutivité et simplicité dans la gestion des données.

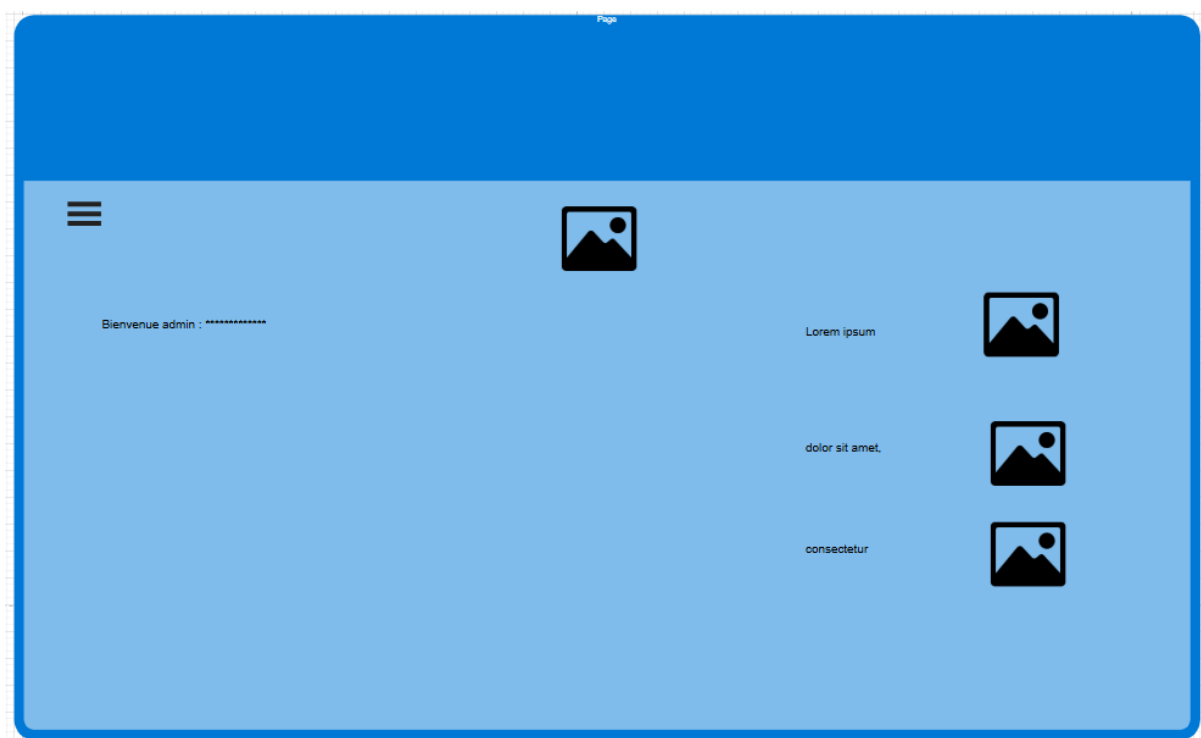
3.3 Zoning :



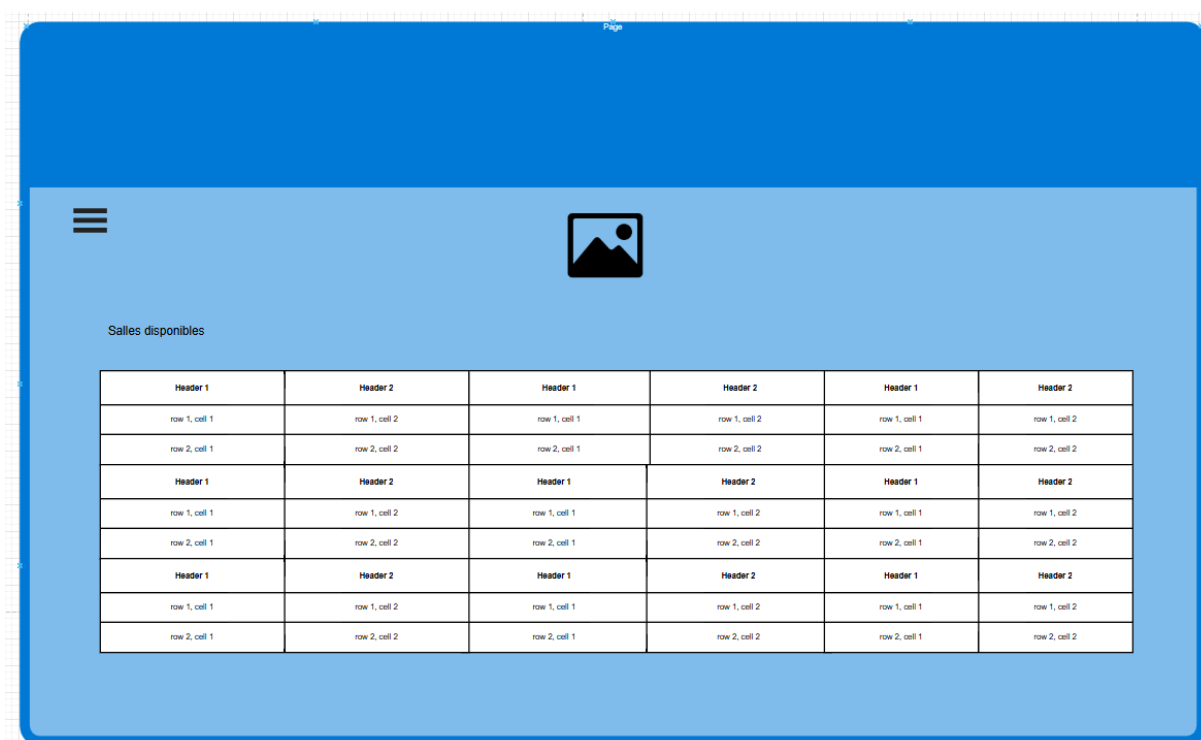
Zoning 2 Visiteur SignIN



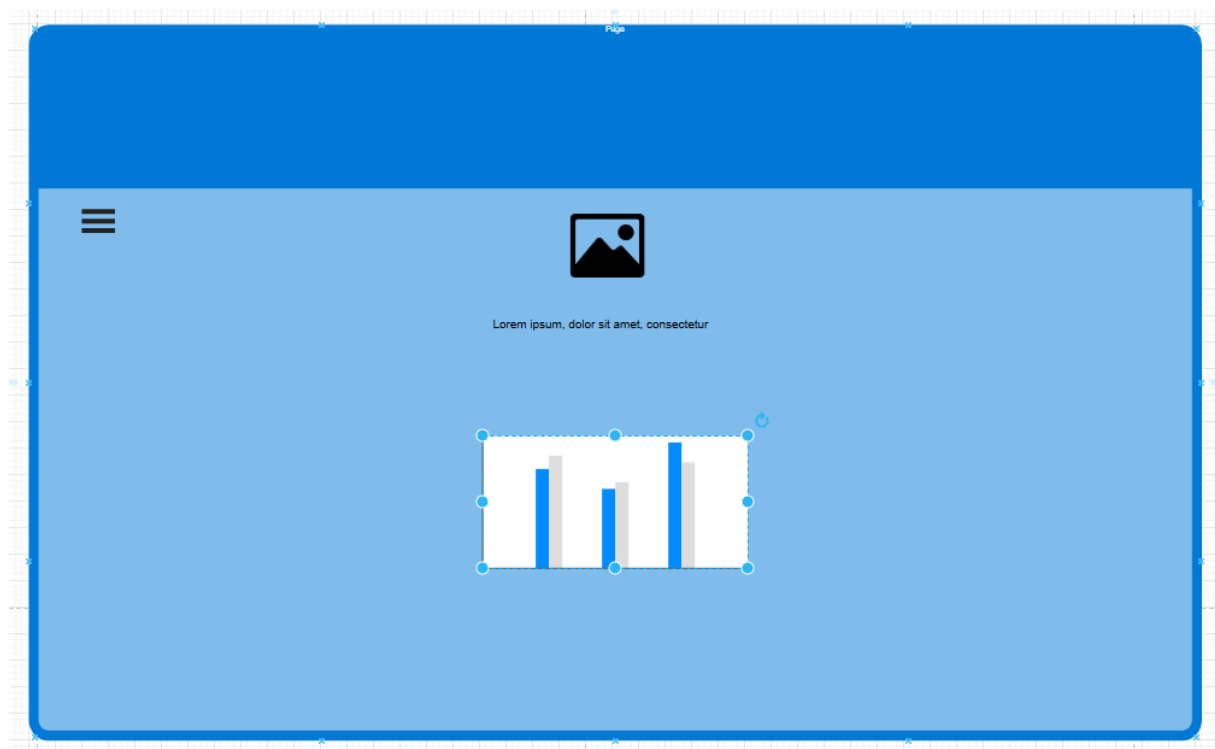
Zoning 1 Visiteur View



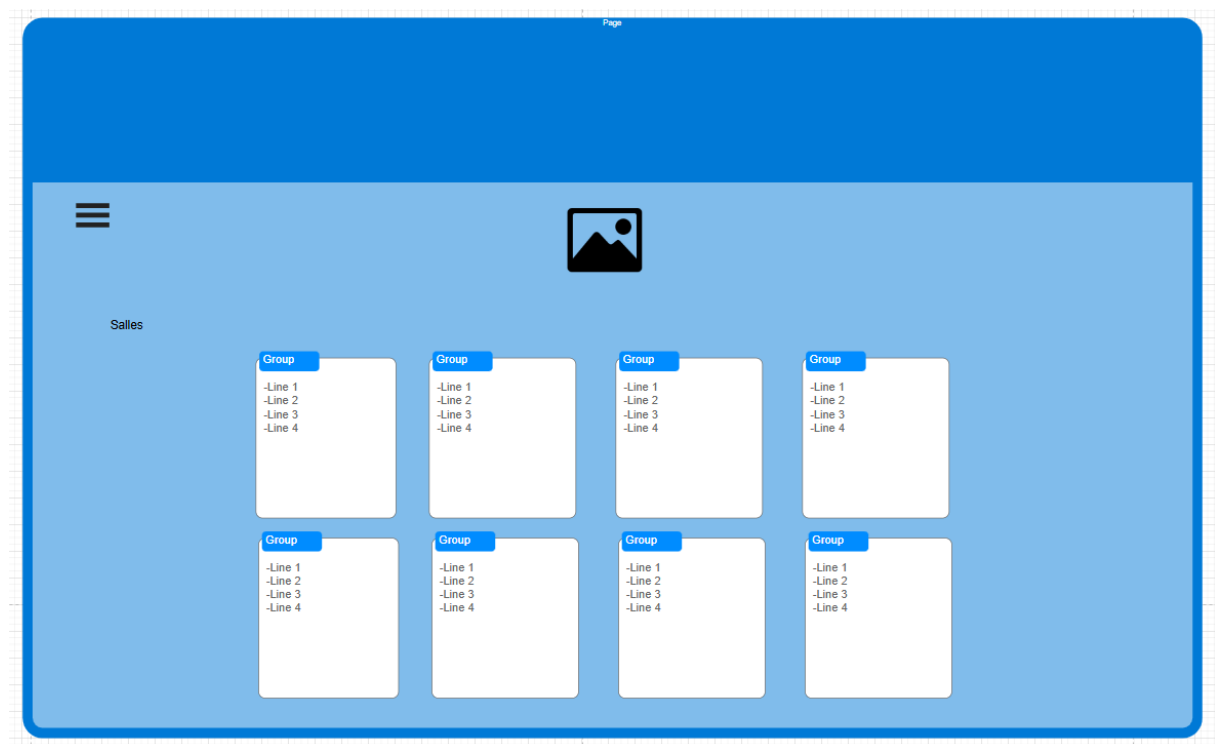
Zoning 4 Admin dashboard



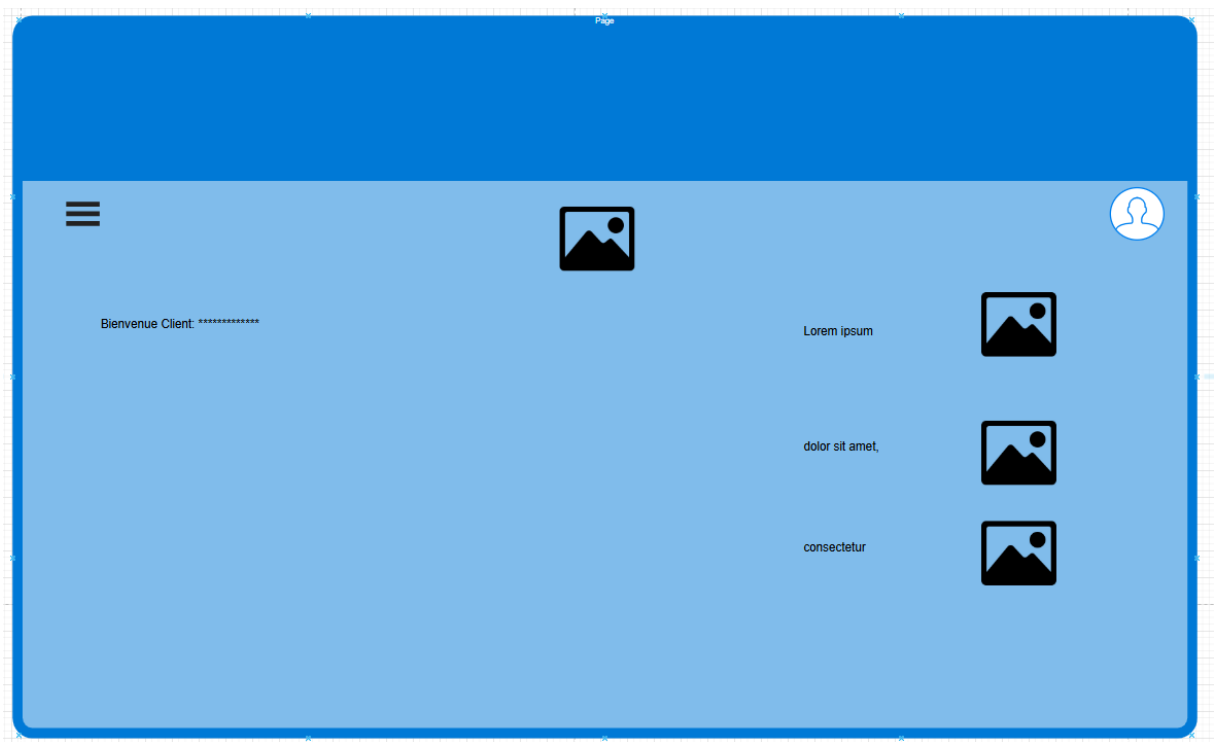
Zoning 3 Admin Salles disponibles



Zoning 5 Admin statistiques



Zoning 6 Client view

*Zoning 7 Client dashboard*

3.4 Stratégie de test

On procède à des tests fonctionnels afin de garantir que les fonctionnalités clés (réservations, affichage des salles, gestion des rôles) fonctionnent comme prévu. Les tests UX sont mis en place pour valider la clarté des wireframes et l'ergonomie générale de l'application.

Des tests de sécurité sont effectués pour garantir la protection des données des utilisateurs et l'intégrité des réservations.

3.4.1 Moyens à mettre en œuvre

Afin d'assurer l'ergonomie et la fiabilité de l'application, plusieurs moyens sont mis en œuvre lors des tests. Des tests manuels sont réalisés sous la supervision du chef de projet, qui suit précisément les scénarios de test définis afin de valider le bon fonctionnement des différentes fonctionnalités. En complément, des retours d'utilisateurs sont recueillis pour évaluer la clarté des wireframes et la fluidité de l'application.

3.4.2 Couverture des tests (tests exhaustifs ou non)

Tests exhaustifs pour les fonctionnalités critiques (authentification, accès aux salles, gestion des réservations, sécurité des données).

Tests partiels pour les aspects UX, basés sur un échantillon d'utilisateurs cibles.

Tests ciblés en sécurité, focalisés sur les règles Firestore, les accès selon les rôles,

et la gestion des sessions. L'objectif n'est pas de tester tous les cas possibles, mais de couvrir les scénarios les plus probables et ceux présentant un risque critique.

3.4.3 Données de test à prévoir

Des comptes utilisateurs fictifs sont mis en place pour évaluer le processus d'authentification et de contrôle des accès selon le rôle (visiteur, client, administrateur).

Ces comptes incluent des cas variés : un compte client actif, un administrateur, un utilisateur avec une tentative d'accès non autorisé, et un email déjà existant pour tester la gestion des doublons.

Des réservations simulées sont créées pour tester le processus complet : sélection de salle, ajout de matériel en option, modification et annulation dans les délais autorisés. Ces réservations permettent aussi de tester les conflits de dates et la disponibilité en temps réel.

Des tests d'affichage sont réalisés pour s'assurer que l'interface est bien optimisée sur différents types d'appareils et résolutions, notamment sur mobile, tablette et desktop. Des tests d'accessibilité sont également prévus afin de vérifier que les éléments importants (disposition, prix, matériel) restent visibles et compréhensibles pour tous.

Scénarios d'erreur sont simulés pour tester la robustesse de l'application : tentative d'accès à une salle protégée sans connexion, modification de l'ID Firestore pour réserver une salle interdite, ou saisie de dates incohérentes dans une réservation.

3.4.4 Testeurs extérieurs éventuels

Les utilisateurs finaux, principalement des étudiants ayant besoin de salles de réunion ou d'événements, participent aux tests UX afin d'évaluer la clarté et l'ergonomie de l'interface. Leurs retours permettent d'identifier les points d'amélioration nécessaires pour assurer une navigation fluide et intuitive.

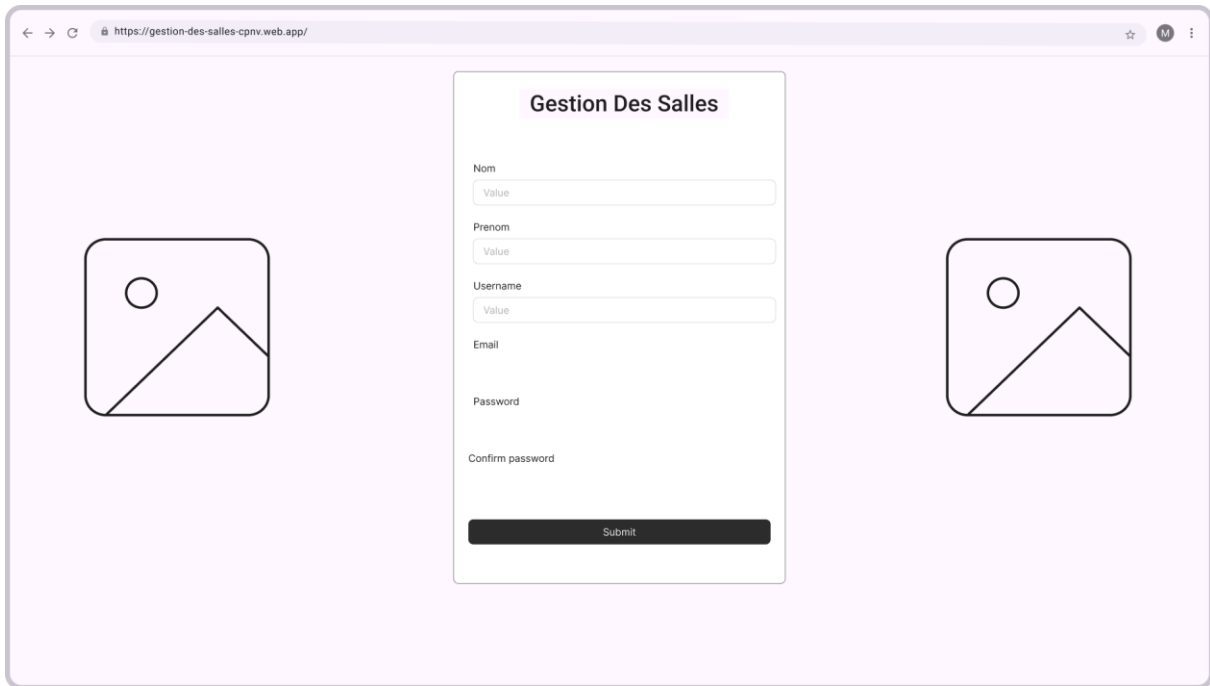
3.5 Planification

Le développement de l'application "Gestion des salles" a été réalisé en adoptant une méthodologie Agile, structurée autour de sprints courts. Cette approche m'a permis de gérer le projet de manière itérative, en intégrant régulièrement des améliorations et en ajustant le développement en fonction des retours obtenus à la fin de chaque sprint. Le choix de la méthode Agile repose sur sa capacité à favoriser la réactivité, à maintenir une progression constante et à garantir la livraison de fonctionnalités concrètes à chaque étape.

Pour la gestion du projet, l'outil IceScrum a été utilisé comme plateforme centrale. Cet outil a permis d'organiser efficacement le backlog, de rédiger les user stories correspondant aux différents profils d'utilisateurs (visiteur, client, administrateur), de planifier les sprints, et de suivre visuellement l'état d'avancement. Il a également facilité la documentation des décisions clés, assurant une coordination claire et une bonne traçabilité tout au long du développement.

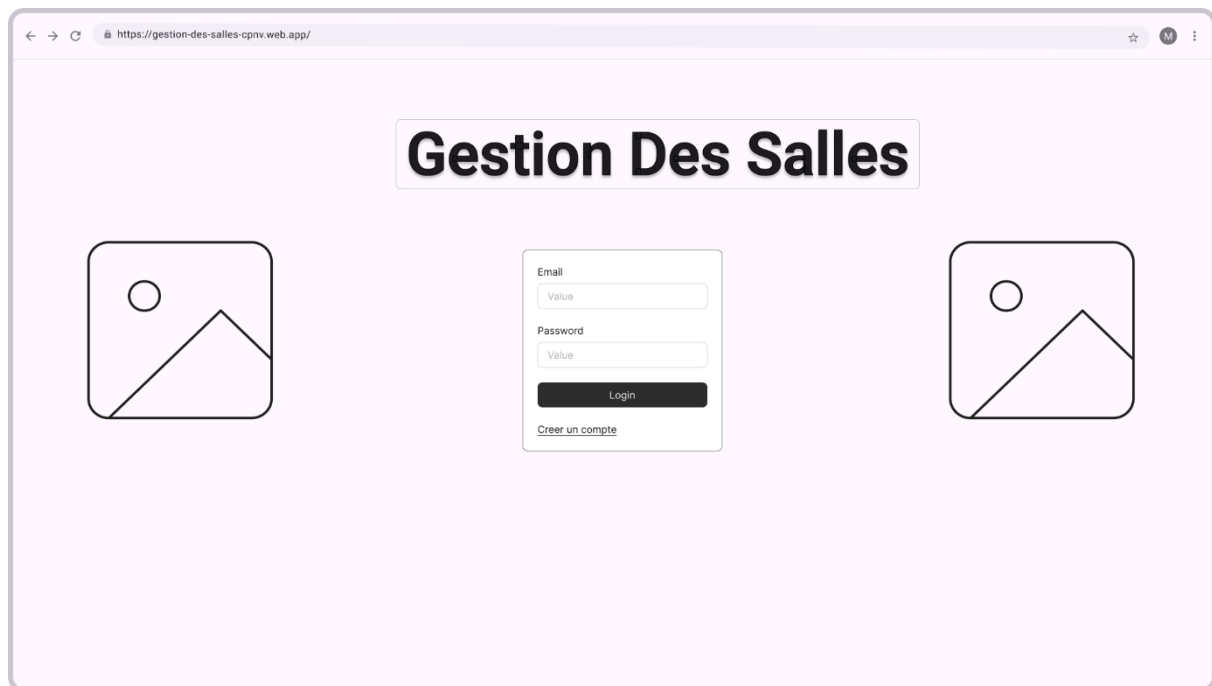
3.6 Dossier de conception

- Architecture et Infrastructure : Le projet est fait avec Angular pour le frontend et Firebase pour le backend. Ce choix permet une intégration fluide avec les services cloud. L'architecture backend repose sur Firestore pour le stockage des données, sans API REST supplémentaire.
- Sécurité et Authentification : L'authentification des utilisateurs est gérée via Firebase Authentication, le service permet une connexion sécurisée avec Google et Email/MDP. Les règles de sécurité Firestore ont été définies pour restreindre l'accès aux données personnelles de chaque utilisateur.
- Expérience Utilisateur (UX/UI) : Des wireframes ont été créés pour toutes les pages principales de l'application, l'app garantit une navigation fluide et intuitive. Un guide de style a été établie pour assurer la cohérence visuelle du projet. Le site web est conçue pour être responsive, afin de fonctionner aussi bien sur desktop.

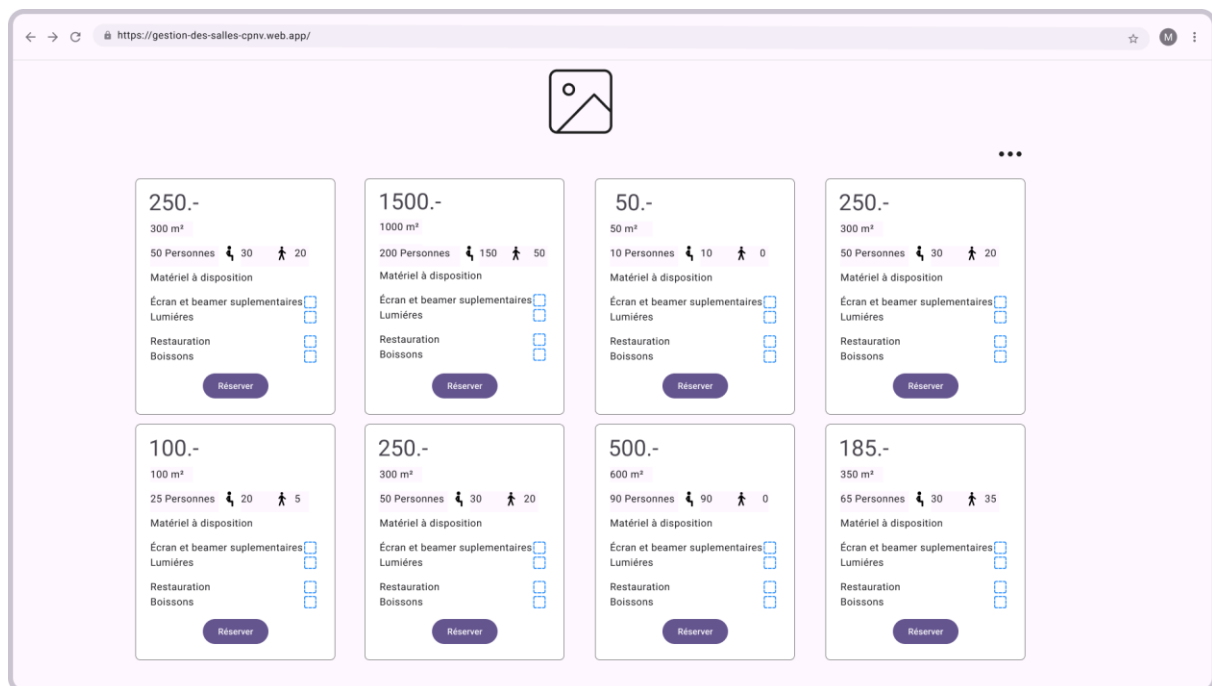


The image shows a web browser window displaying a sign-in form. The browser's address bar shows the URL `https://gestion-des-salles-cpnr.web.app/`. The form is centered on a light purple background and is titled "Gestion Des Salles". It contains the following fields: "Nom" (Name), "Prenom" (First Name), "Username", "Email", "Password", and "Confirm password". Each of the first four fields has a placeholder text "Value". Below the "Confirm password" field is a dark grey "Submit" button. On either side of the form, there is a square placeholder icon for a profile picture, featuring a circle and a mountain silhouette.

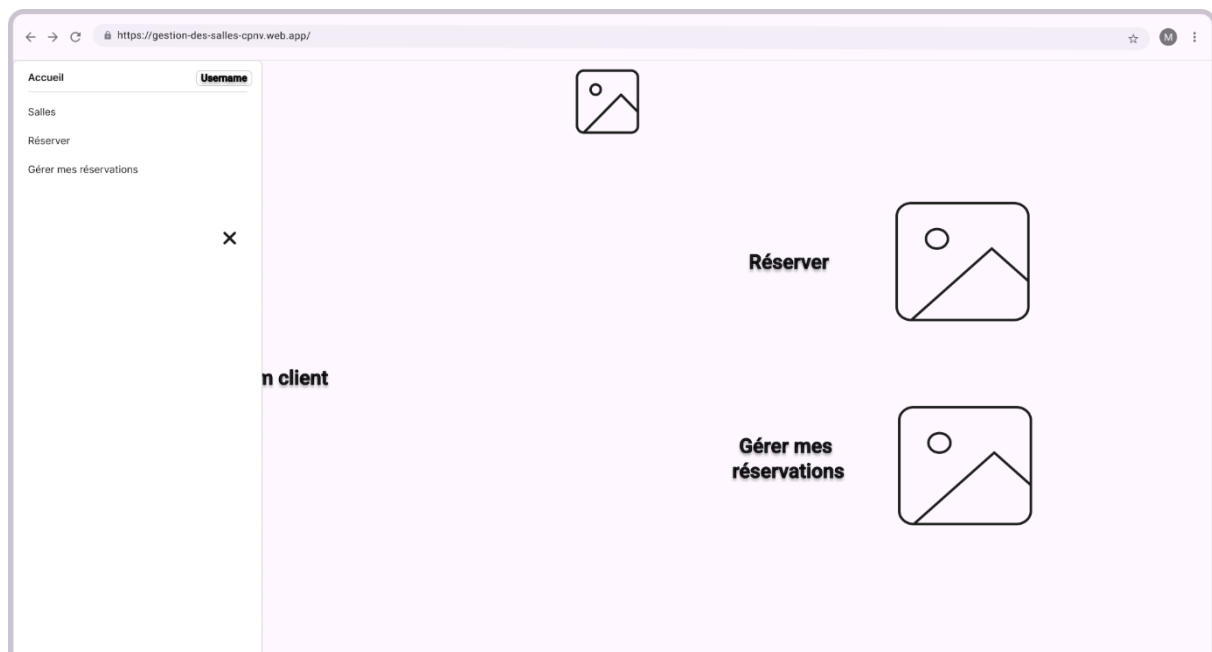
Wireframes 1 Sign-in



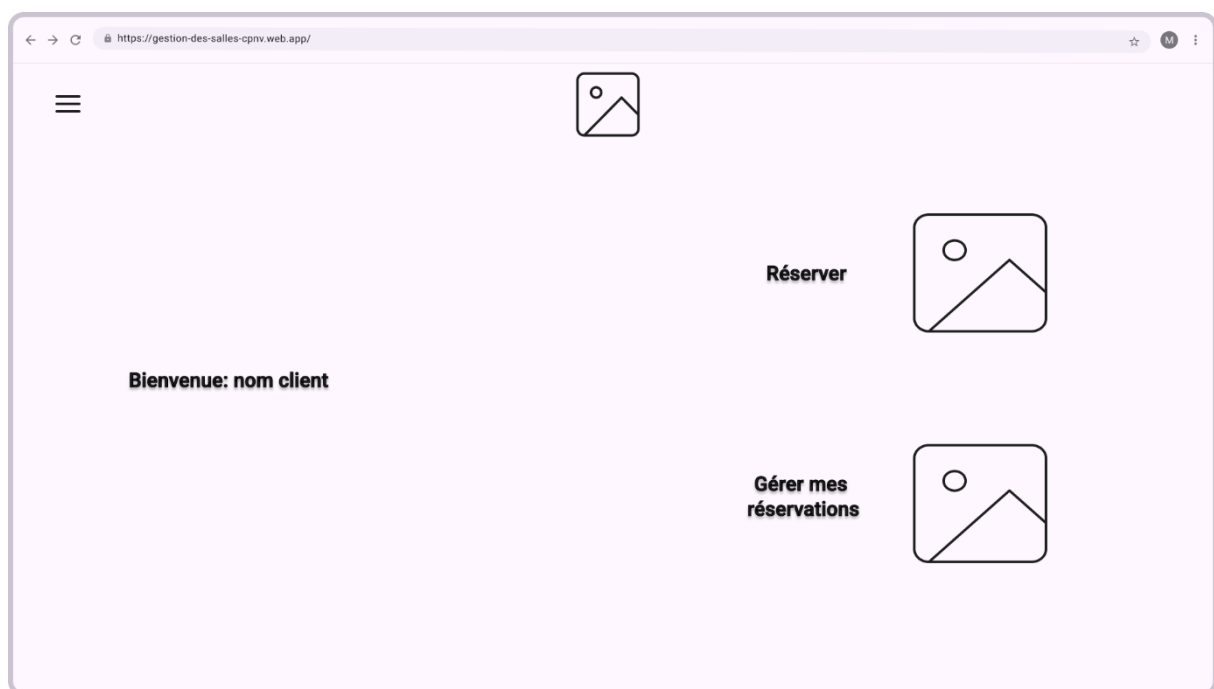
Wireframes 3 Log-in



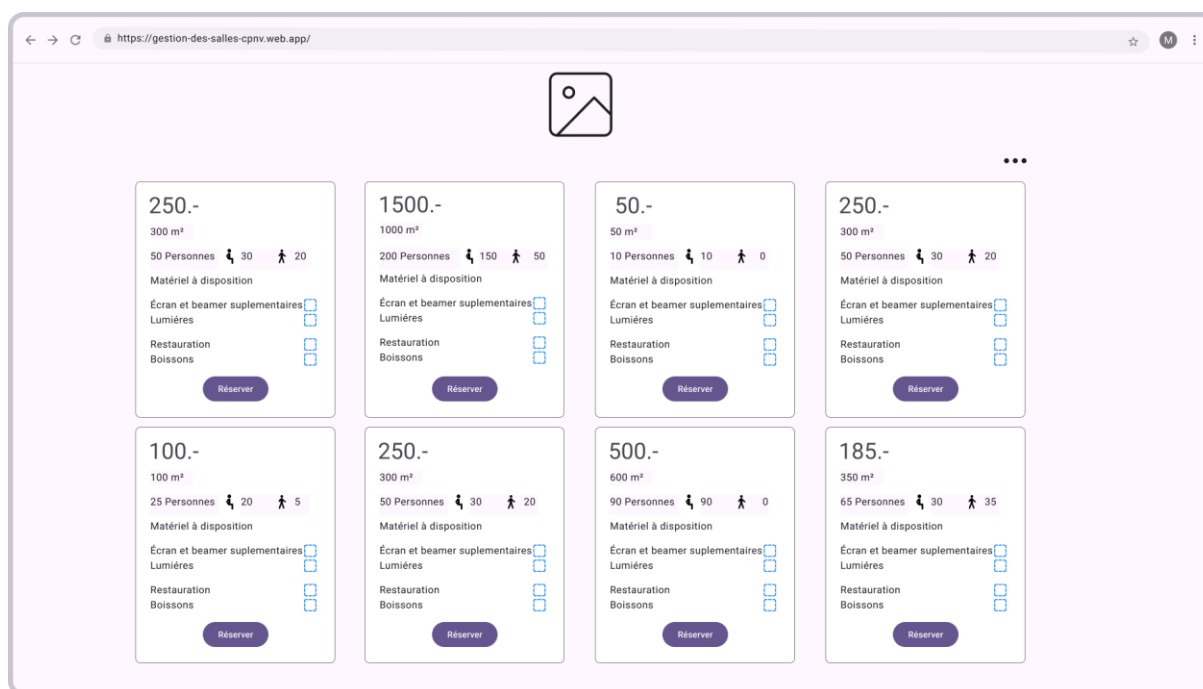
Wireframes 2 Visiteur View



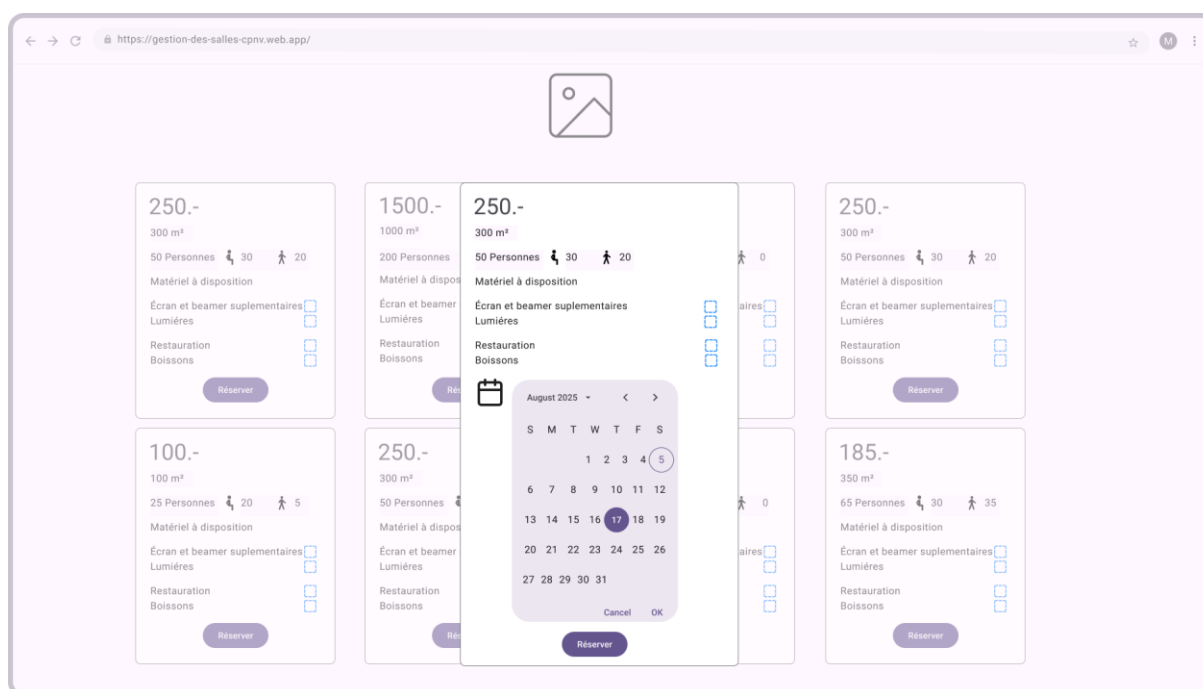
Wireframes 4 Client menu dashboard



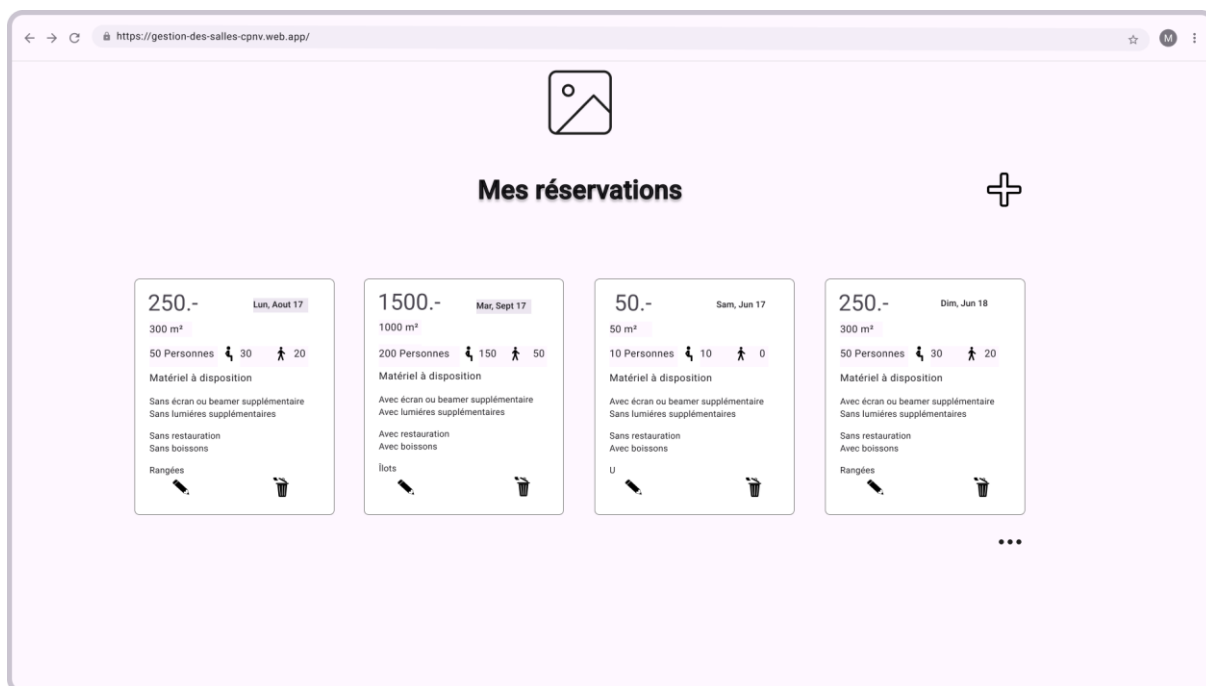
Wireframes 5 client Dashboard



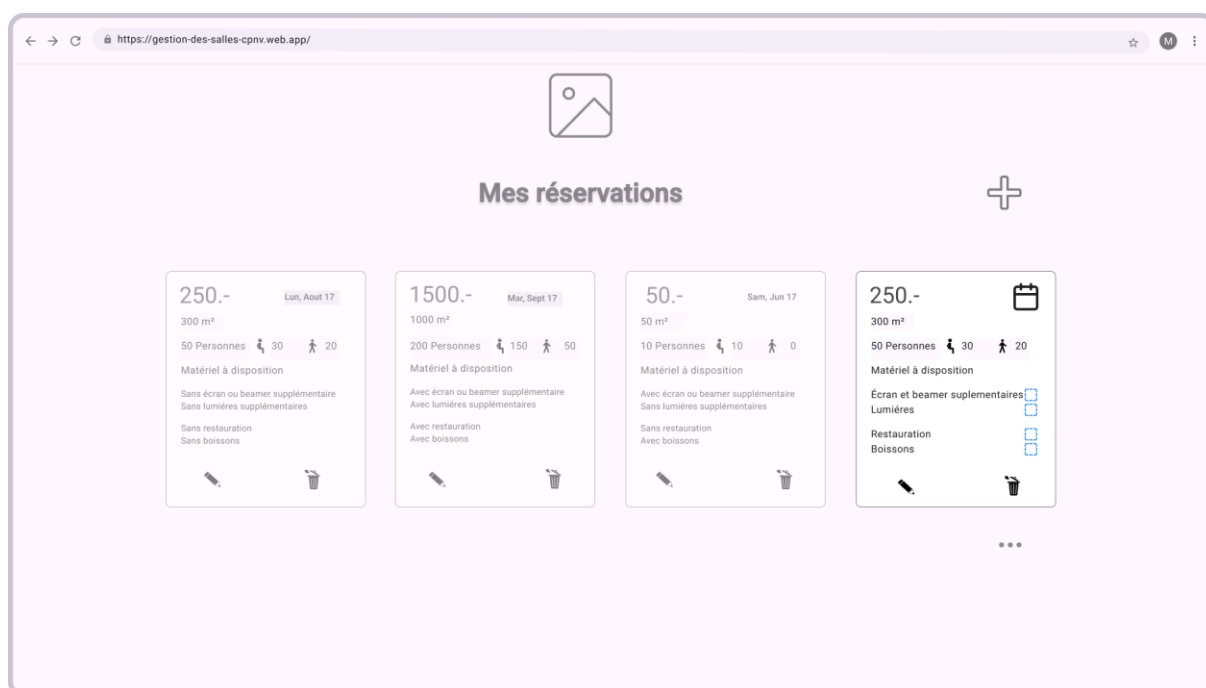
Wireframes 6 Client salles



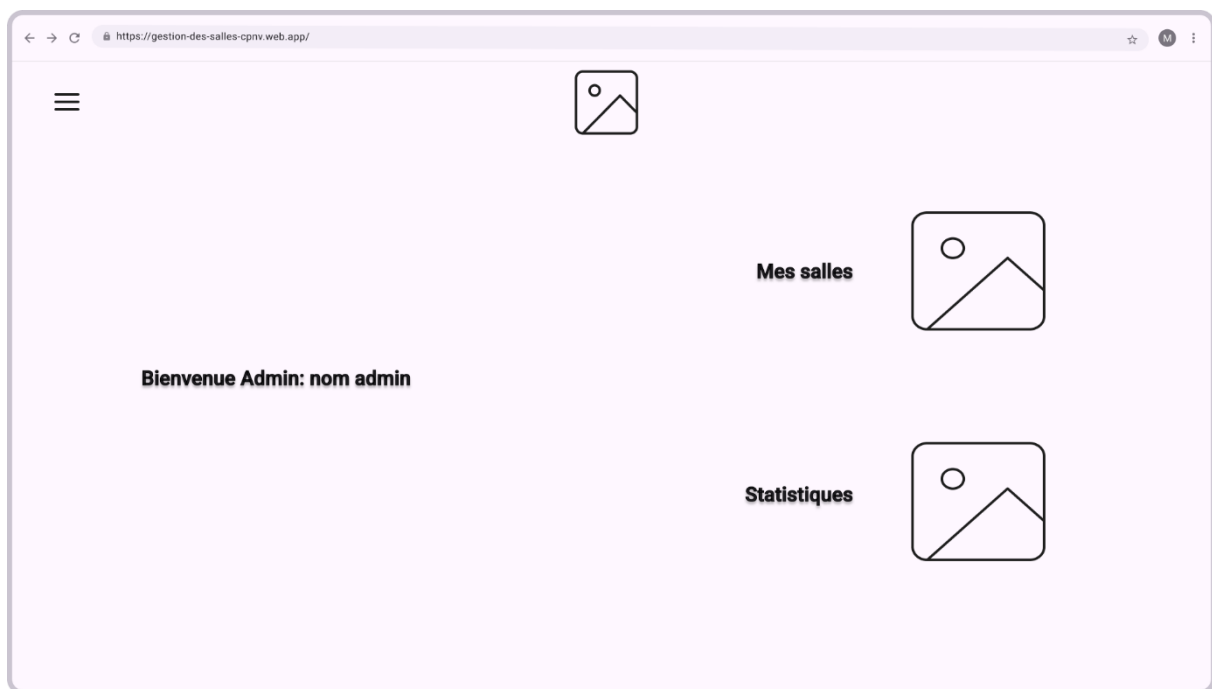
Wireframes 7 client réserve



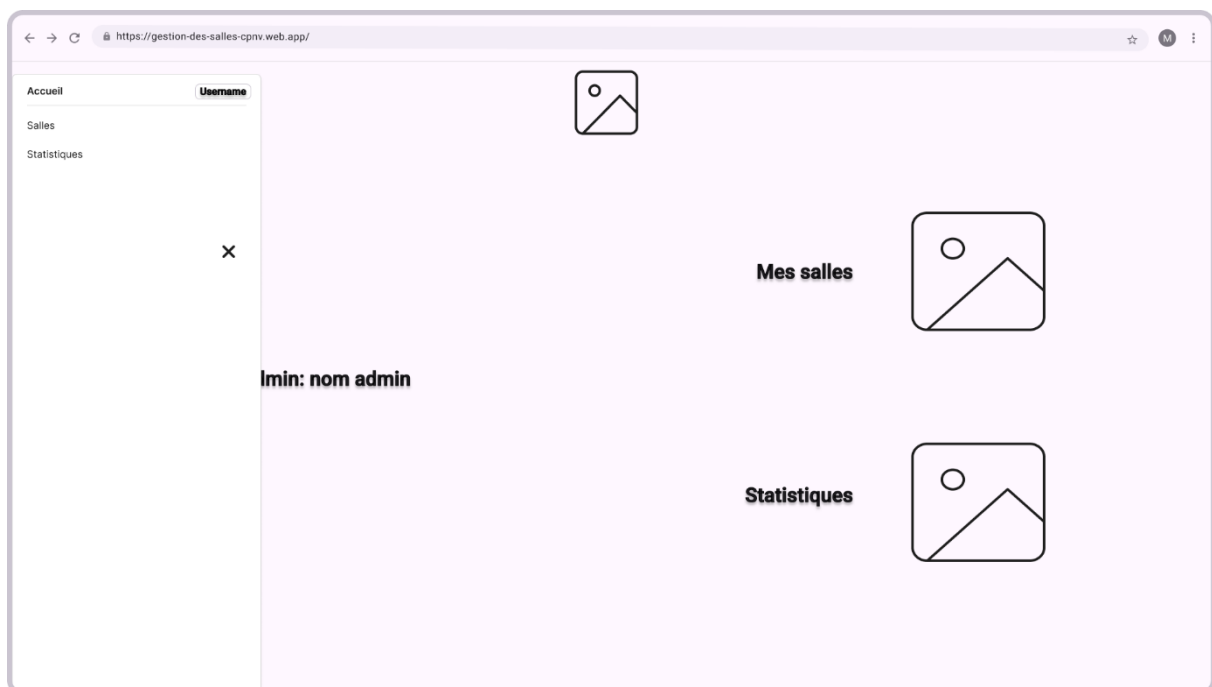
Wireframes 9 client réservations



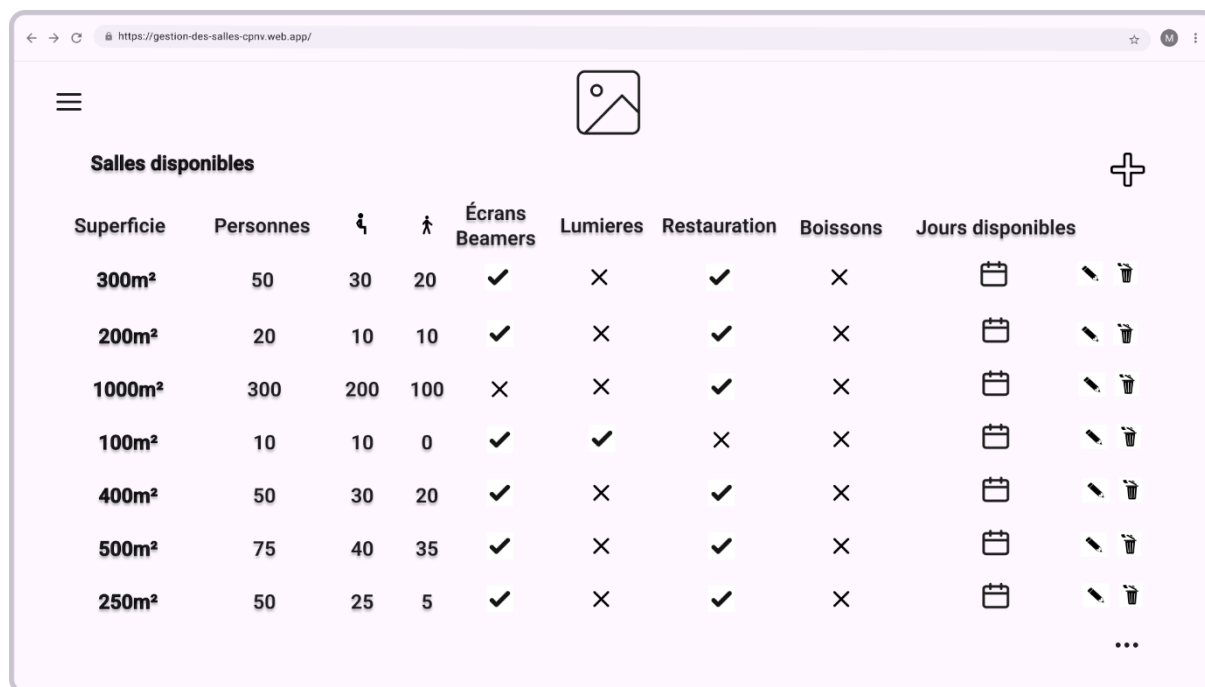
Wireframes 8 client modification réservations



Wireframes 10 admin dashboard



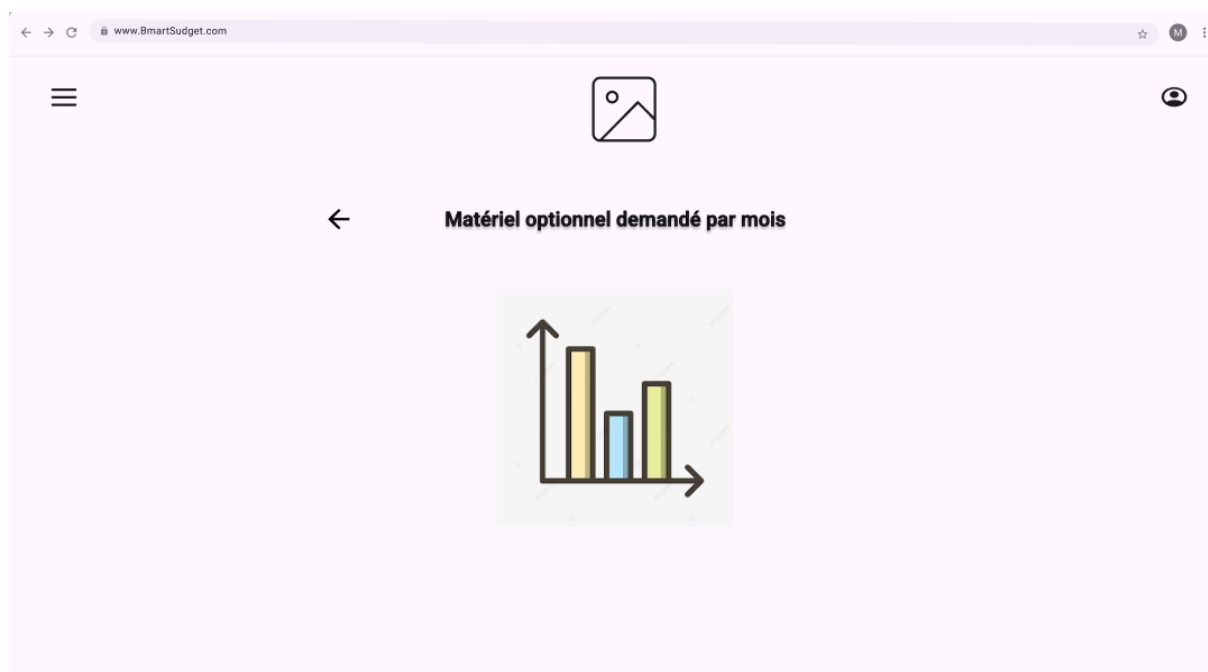
Wireframes 11 admin dashboard menu



Salles disponibles

Superficie	Personnes	Écrans	Beamers	Lumieres	Restauration	Boissons	Jours disponibles
300m ²	50	30	20	✓	×	✓	×
200m ²	20	10	10	✓	×	✓	×
1000m ²	300	200	100	×	×	✓	×
100m ²	10	10	0	✓	✓	×	×
400m ²	50	30	20	✓	×	✓	×
500m ²	75	40	35	✓	×	✓	×
250m ²	50	25	5	✓	×	✓	×

Wireframes 12 admin gestion salles



Wireframes 13 admin statistiques



Wireframes 14 admin statistiques (1)

- Gestion des Données et Performance : Une stratégie de stockage a été mise en place en structurant les collections Firestore pour optimiser les requêtes. Cela permet de minimiser les coûts d'opération tout en assurant un accès rapide aux données. Des tests de cohérence ont été réalisés pour valider la structure de la base de données.

🏠 > users > b2s95CMmTSW.			☁️ Más funciones en Google Cloud ▼	
⚙️ (default)	📁 users	⋮	📄 b2s95CMmTSWS06tYtu0b	⋮
+ Iniciar colección	+ Agregar documento		+ Iniciar colección	
equipment	b2s95CMmTSWS06tYtu0b	>	+ Agregar campo	
reservations			email: "santiago.messer@eduvaud.ch"	
rooms			first_name: "Santiago"	
users		>	last_name: "Messer"	
			role: "admin"	(cadena) ✎ 🗑️

NoSQL 1 users collection

rooms > ECU4xbLshA2wl.			Más funciones en Google Cloud
(default)	rooms	ECU4xbLshA2wl08DSasM	
+ Iniciar colección	+ Agregar documento	+ Iniciar colección	
equipment	ECU4xbLshA2wl08DSasM	+ Agregar campo	
reservations		area: "120"	
rooms		equipment_description: "Tables, chairs, wall screen"	
users		name: "Multipurpose Room 1"	
		price_per_day: 250	
		seated_places: 35	
		total_seats: 50	

NoSQL 4 rooms collection

equipment > rh9Axt0qW8WO..			Más funciones en Google Cloud
(default)	equipment	rh9Axt0qW8WOYyooH9up	
+ Iniciar colección	+ Agregar documento	+ Iniciar colección	
equipment	rh9Axt0qW8WOYyooH9up	+ Agregar campo	
reservations		extra_price: 30	
rooms		included: false	
users		name: "4K Beamer"	

NoSQL 3 equipment collection

reservations > RyGZ9kWzKlI2U.			Más funciones en Google Cloud
(default)	reservations	RyGZ9kWzKlI2U74wds63	
+ Iniciar colección	+ Agregar documento	+ Iniciar colección	
equipment	RyGZ9kWzKlI2U74wds63	equipment	
reservations		+ Agregar campo	
rooms		disposition: "rangées"	
users		end_date: 12 de abril de 2025, 3:51:31 p.m. UTC+2	
		room_id: /rooms/ECU4xbLshA2wl08DSasM	
		start_date: 11 de abril de 2025, 10:49:05 a.m. UTC+2	
		status: "confirmed"	
		user_id: /users/b2s95CMmTSWS06tYtu0b	

NoSQL 2 reservations

 > reservations > RyGZ9kWzKlI2U... > equipment > nz3CwT8v2WRT. Más funciones en Google Cloud		
 RyGZ9kWzKlI2U74wds63	 equipment	 nz3CwT8v2WRTtM4whEyR
+ Iniciar colección	+ Agregar documento	+ Iniciar colección
equipment >	nz3CwT8v2WRTtM4whEyR >	+ Agregar campo
+ Agregar campo		equipment_id: /equipment/rh9Axt0qW8WOYooH9up quantity: 2
end_date: 12 de abril de 2025, 3:51:31 p.m. UTC+2 room_id: /rooms/ECU4xbLshA2wl08DSasM start_date: 11 de abril de 2025, 10:49:05 a.m. UTC+2 status: "confirmed" user_id: /users/b2s95CMmTSWS06tYtu0b		

NoSQL 5 equipment ref

Collection : users

Chaque document représente un utilisateur unique, identifié par un ID généré automatiquement. Les champs stockés sont :

email : Adresse email de l'utilisateur.

first_name : Prénom de l'utilisateur.

last_name : Nom de famille de l'utilisateur.

role : Rôle de l'utilisateur (admin, client, visiteur), utilisé pour déterminer ses droits d'accès dans l'application.

Collection : rooms

Contient les informations liées aux salles disponibles à la location. Chaque salle est définie par les champs suivants :

name : Nom de la salle.

area : Superficie en mètres carrés.

total_seats : Nombre total de places disponibles.

seated_places : Nombre de places assises avec tables.

equipment_description : Description du matériel inclus dans la salle.

price_per_day : Prix de location journalier.

Collection : equipment

Stocke les éléments matériels pouvant être ajoutés en option à une réservation.
Les champs sont :

name: Nom du matériel (ex. Beamer 4K, lumière d'appoint).

included : Booléen indiquant si le matériel est inclus par défaut dans une salle.

extra_price : Supplément tarifaire par unité si le matériel est optionnel.

Collection : reservations

Gère toutes les réservations effectuées dans l'application. Chaque réservation est liée à un utilisateur et à une salle spécifique. Les champs incluent :

user_id : Référence vers l'utilisateur ayant effectué la réservation.

room_id : Référence vers la salle réservée.

start_date : Date et heure de début de la réservation.

end_date : Date et heure de fin.

status : État de la réservation (true pour active, false pour annulée).

equipment : Objet contenant les équipements optionnels ajoutés à la réservation.
Chaque sous-document contient :

equipment_id : Référence vers un matériel.

quantity : Quantité demandée.

disposition : distribution de la salle (en îlots, rangées ou en U).

Cette structure de base de données permet une lecture rapide, une évolutivité naturelle et une intégration simple avec les composants de l'application Angular. Elle favorise également un développement modulaire, où chaque fonctionnalité peut évoluer indépendamment.

3.6.1 Règles Stockage

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Utilisateurs : chaque utilisateur ne peut lire/modifier que son propre document
    match /users/{userId} {
      allow read, update, delete: if request.auth != null && request.auth.uid == userId;
      allow create: if request.auth != null;
    }

    // Salles : tout le monde peut lire, seuls les administrateurs peuvent modifier
    match /rooms/{roomId} {
      allow read: if true; // accès public
      allow write: if isAdmin();
    }

    // Matériel : même logique que les salles
    match /equipment/{equipmentId} {
      allow read: if true;
      allow write: if isAdmin();
    }

    // Réservations : les utilisateurs peuvent uniquement gérer leurs propres
    réservations
    match /reservations/{reservationId} {
      allow create: if request.auth != null;
      allow read, update, delete: if request.auth != null && resource.data.user_id ==
      ('/users/' + request.auth.uid);
    }

    // Sous-collection : matériel lié à une réservation
    match /reservations/{reservationId}/equipment/{equipId} {
      allow read, write: if isOwnerOfReservation(reservationId);
    }

    // === Fonctions utilitaires ===
    function isAdmin() {
      return request.auth != null &&
        get(/databases/{database}/documents/users/{request.auth.uid}).data.role
      == "admin";
    }

    function isOwnerOfReservation(resId) {
      return request.auth != null &&
        get(/databases/{database}/documents/reservations/{resId}).data.user_id
      == ('/users/' + request.auth.uid);
    }
  }
}
```



```

}
}
}

```

Collection	Visiteur anonyme	Utilisateur connecté	Administrateur
users	Non	Lecture/écriture de son document uniquement	Oui pour soi-même
rooms	Oui	Oui	Modification possible
equipment	Oui	Oui	Modification possible
reservations	Non	Lecture et écriture de ses propres réservations	Oui si adapté
reservations/*/equipment	Non	Accès si propriétaire de la réservation	Oui si propriétaire

4 Réalisation

4.1 Dossier de réalisation

Répertoires où le logiciel est installé :

Le projet est hébergé et exécuté dans un environnement de développement local

```

|-/src
|- /app
  |--/services      -> Contient les fichiers avec interactions Firebase.
  |
  |----- app.component.ts -> Composant principal.
  |----- app.routes.ts   -> Fichier de configuration des routes.
  |----- app.config.ts   -> Configuration de l'application.
  |--/environments    -> Contient les config d'environnement (dev/prod).
  |--/auth
  |--/models (prochaine sprint)
  |--/dashboard

```

```
|
|----- index.html      -> Fichier HTML principal.
|----- main.ts         -> Point d'entrée Angular.
|----- styles.css       -> Styles globaux.
```

Les fichiers principaux du projet sont organisés de la manière suivante :

index.html → Fichier principal de l'application web.
main.ts → Point d'entrée principal de l'application Angular.
styles.css → Fichier de style global.

app/ → Dossier principal du code source.

app.component.ts, app.component.html, app.component.css, app.component.spec.ts

app.config.ts → Configuration principale de l'application

app.routes.ts → Définition des routes principales (même si elles ne sont pas encore utilisées)

environments/environments.ts → Configuration des environnements (dev/prod)

Dossiers vides avec leur structure de base

auth/ → Gestion de l'authentification (login, register, etc.).

services/ → Préparation des services (communication avec Firebase).

models/ → Fichiers de modélisation des données (user.models.ts, transactions.models.ts).

dashboard/ → Base pour l'interface principale.

Système d'exploitation utilisé :

Développement : Windows 11

Outils logiciels utilisés :

Framework frontend :

```
(
@angular-devkit/architect    0.1901.6
@angular-devkit/build-angular 19.1.6
@angular-devkit/core         19.1.6
@angular/cli                 19.1.6
@angular/fire                 19.0.0
```



```
@angular/material      19.1.3
rxjs                   7.8.1
typescript             5.5.4
zone.js                0.15.0
)
```

Base de données : Firebase Firestore (NoSQL)
Authentification : Firebase Auth
Gestion de projet : IceScrum
Prototypage : Figma
Backend : Firebase Cloud Functions (Node.js 18)
Statistiques : Chart.js + ng2-charts

- **Description exacte du matériel :**

Processeur : Intel(R) Core(TM) i5-9500 CPU @ 3.00GHz 3.00 GHz
RAM : 16 Go
Stockage : SSD 512 Go
OS : Windows 11

- **Programmation et scripts**

Node.js et npm :
node -v # Vérifie la version de Node.js

Installer Angular CLI et TypeScript
npm install -g @angular/cli

Initialiser le projet Angular:
ng new GestionDesSalles
cd GestionDesSalles

📌 Aussi on peut le récupérer depuis GitHub:
git clone git@github.com:santimesser/GestionDesSalles.git
cd GestionDesSalles

Installer AngularFire et Firebase
ng add @angular/fire

Installer Firebase Authentication
npm install firebase

Ajoute Chart.js pour afficher les statistiques financières :
npm install chart.js
npm install ng2-charts

Installer RxJS pour gérer les flux de données :
(en general RxJS est déjà inclus avec Angular mais il faut s'assurer d'avoir la dernière version)
npm install rxjs

Lancer le projet en mode développement :
ng serve --open

4.1.1 Authentification et gestion du profil

4.1.1.1 Objectif du Sprint

L'objectif principal de ce sprint était de mettre en place un système d'authentification sécurisé et convivial, permettant aux utilisateurs de s'inscrire, de se connecter, et de définir un identifiant utilisateur unique pour accéder aux fonctionnalités de l'application.

4.1.1.2 Travail réalisé

1. Inscription et connexion des utilisateurs

- Création d'un compte via Google ou à l'aide d'une adresse e-mail unique et d'un mot de passe sécurisé.
- Gestion des erreurs courantes avec messages explicites (e-mail déjà utilisé, mot de passe faible, etc.).
- Connexion sécurisée et redirection vers une page de configuration initiale après authentification.
- Vérification de l'adresse e-mail via l'envoi automatique d'un lien de validation.
- Fonctionnalité "mot de passe oublié" avec envoi d'un lien de réinitialisation par e-mail.

2. Personnalisation du profil utilisateur

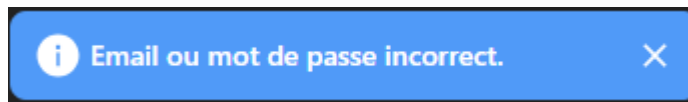
- Lors de la première connexion, l'utilisateur doit choisir un nom d'utilisateur (username).
- Ce champ est obligatoire : si l'utilisateur ne remplit pas un username valide (différent de vide), il ne peut pas continuer.
- Le nom d'utilisateur est stocké dans Firestore et utilisé pour identifier l'utilisateur dans le reste de l'application.

3. Sécurisation et gestion des sessions

- Utilisation de Firebase Authentication pour gérer les connexions, déconnexions et la vérification des rôles.
- Les routes protégées ne sont accessibles qu'aux utilisateurs authentifiés, avec gestion des droits selon le rôle (admin, client, visiteur).
- Mise en place d'un bouton de déconnexion avec invalidation immédiate de la session active.

4. Gestion des erreurs et validation utilisateur

- Affichage de messages d'erreur clairs en cas de mauvais identifiants ou d'accès non autorisé.
- Blocage de l'accès aux sections sensibles tant que l'e-mail n'a pas été vérifié ou qu'un username valide n'a pas été défini.



Gestion des erreurs 1

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

5.2 Sources – Bibliographie

5.3 Journal de travail

Date ▼	Depart ▼	Fin ▼	Temps ▼	Cours ▼	Tâche ▼	Titre ▼	Commentaire
07.04.2025	08.15 h	08.45 h	00:30	TPI	Analyse	Cahier des charges	Lecture du cahier des charges
07.04.2025	08.45 h	08.50 h	00:05	TPI	Planification	Creation journal de travail et IceScrum	Creation du journal de travail et IceScrum
07.04.2025	08.50 h	09.00 h	00:10	TPI	Planification	Creation Git-hub	Creation Git-hub
07.04.2025	09.00 h	09.50 h	00:50	TPI	Planification	Planning TPI	Découpage de tâches et compréhension du TPI
07.04.2025	10.10 h	10.31 h	00:21	TPI	Planification	Reunion avec le expert	Reunion avec le expert
07.04.2025	10.31 h	12.00 h	01:29	TPI	Planification	Definition des sprint goals	Définition des sprint goals et finalisation du PDF lisible
07.04.2025	12.00 h	12.30 h	00:30	TPI	Planification	Creation stories Sprint 1	Creation des stories de mon sprint 1
07.04.2025	13.20 h	13.56 h	00:36	TPI	Planification	Creation tasks et tests Sprint 1	Creation tasks et tests d'acceptation Sprint 1
07.04.2025	13.56 h	14.10 h	00:14	TPI	Planification	Creation des features	Creation des features pour l'application
07.04.2025	14.10 h	14.30 h	00:20	TPI	Planification	Angular update et mise en place	Update angular (version: 19.2.6)
07.04.2025	14.30 h	15.20 h	00:50	TPI	Planification	Mise en place firebase	Creation du service hosting, cloud (database) et authentification
07.04.2025	15.20 h	15.40 h	00:20	TPI	Planification	Email	Email Planification initial
08.04.2025	08.15 h	08.59 h	00:44	TPI	Analyse	Wireframes et zoning	Création des premières ébauches du site
08.04.2025	08.59 h	09.30 h	00:31	TPI	Analyse	Wireframes et zoning	Création wireframes site Visiteur
08.04.2025	09.30 h	09.50 h	00:20	TPI	Analyse	Wireframes et zoning	Création wireframes site Client
08.04.2025	10.07 h	10.35 h	00:28	TPI	Analyse	Wireframes et zoning	Création wireframes site Client
08.04.2025	10.35 h	10.50 h	00:15	TPI	Analyse	Wireframes et zoning	Création wireframes site Administrateur
08.04.2025	13.20 h	14.05 h	00:45	TPI	Analyse	Wireframes et zoning	Création wireframes site Administrateur
08.04.2025	14.11 h	16.05 h	01:54	TPI	Documentation	Documentation	Documentation
08.04.2025	16.05 h	16.40 h	00:35	TPI	Analyse	MCD	Creation du MCD et entites
09.04.2025	08.15 h	09.27 h	01:12	TPI	Analyse	MCD	Finalization MCD
09.04.2025	09.27 h	09.50 h	00:23	TPI	Analyse	NoSQL	Creation database NoSQL
09.04.2025	10.05 h	10.37 h	00:32	TPI	Analyse	NoSQL	Continuation et finalization database NoSQL
09.04.2025	10.37 h	11.21 h	00:44	TPI	Documentation	Documentation	Documentation
09.04.2025	11.21 h	12.00 h	00:39	TPI	Documentation	Documentation	Documentation
09.04.2025	12.00 h	12.30 h	00:30	TPI	Planification	Creation stories Sprint 2	Creation des stories, task et tests d'acceptation de mon sprint 2
09.04.2025	13.20 h	14.20 h	01:00	TPI	Planification	Creation stories Sprint 3	Creation des stories, task et tests d'acceptation de mon sprint 3
09.04.2025	14.20 h	14.20 h	00:00	TPI	Planification	Fin Sprint 1	Fin Sprint 1
			00:00				



5.4 Cahier des charges

5.5 Manuel d'Utilisation

5.6 Archives du projet